

LAB 2

Containerization & Kubernetes Orchestration

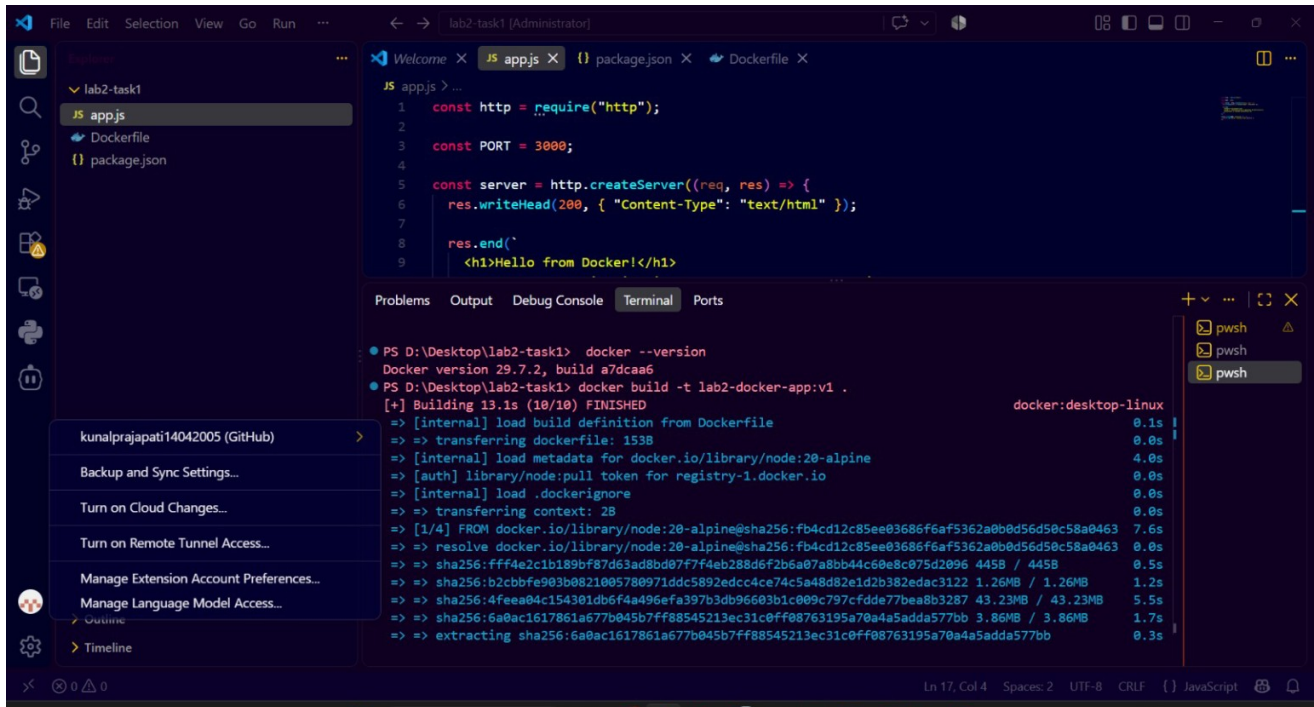
Task 1 — Containerize a Sample Application Using Docker

Student	Kunal Prajapati
Application	Node.js HTTP sample application
Application Port	3000
Docker Image	lab2-docker-app:v1
Container	lab2-container

This corrected report uses the original execution screenshots and places each screenshot under the section it actually demonstrates. No replacement or fabricated evidence has been added.

1. Sample Application and Docker Image Build

The Node.js application uses the built-in HTTP module, listens on port 3000, and returns the lab HTML response. The project contains app.js, Dockerfile and package.json.



The screenshot shows the Visual Studio Code interface with a project named 'lab2-task1'. The file explorer on the left shows 'app.js', 'Dockerfile', and 'package.json'. The editor displays the 'app.js' file with the following code:

```
1 const http = require("http");
2
3 const PORT = 3000;
4
5 const server = http.createServer((req, res) => {
6   res.writeHead(200, { "Content-Type": "text/html" });
7
8   res.end(`
9     <h1>Hello from Docker!</h1>

```

The terminal window shows the execution of the following commands:

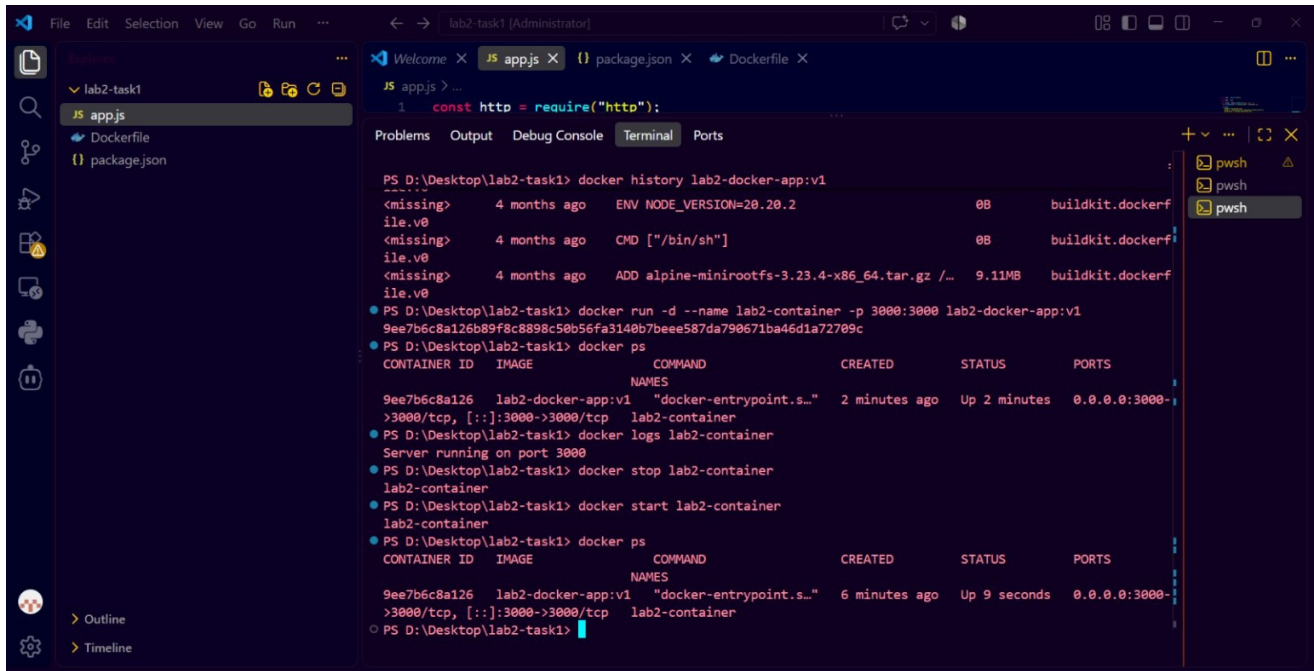
```
PS D:\Desktop\lab2-task1> docker --version
Docker version 29.7.2, build a7dca66
PS D:\Desktop\lab2-task1> docker build -t lab2-docker-app:v1 .
[+] Building 13.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 153B 0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine 4.0s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/4] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a0463 7.6s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a0463 0.0s
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 445B / 445B 0.5s
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 1.26MB / 1.26MB 1.2s
=> => sha256:4f6ea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 43.23MB / 43.23MB 5.5s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 1.7s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.3s
```

Figure 1. Original VS Code and terminal evidence showing the Node.js application and successful **docker build -t lab2-docker-app:v1 .** command.

The terminal shows the Docker build completing successfully from the Node 20 Alpine base image.

2. Docker Image and Layer Documentation

The built image was inspected using **docker history lab2-docker-app:v1**. The history displays application-specific Dockerfile layers such as WORKDIR, COPY, EXPOSE and CMD, together with inherited Node/Alpine layers.



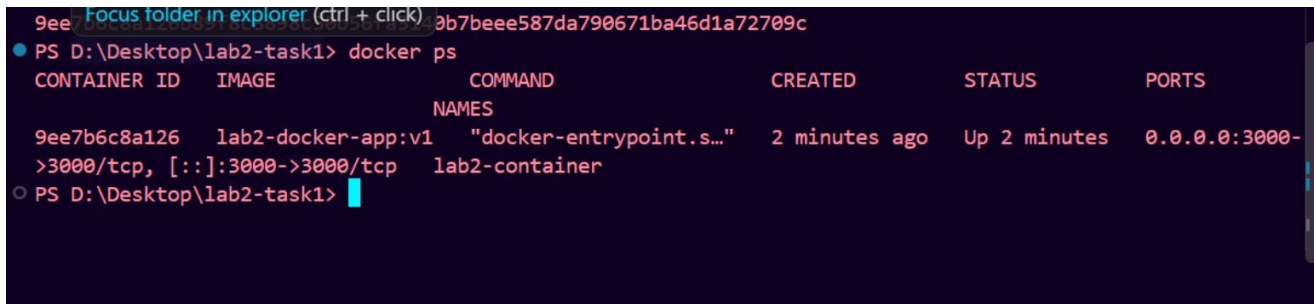
```
PS D:\Desktop\lab2-task1> docker history lab2-docker-app:v1
<missing> 4 months ago  ENV NODE_VERSION=20.20.2  0B  buildkit.dockerf
ile.v0
<missing> 4 months ago  CMD ["/bin/sh"]  0B  buildkit.dockerf
ile.v0
<missing> 4 months ago  ADD alpine-minirootfs-3.23.4-x86_64.tar.gz /...  9.11MB  buildkit.dockerf
ile.v0
• PS D:\Desktop\lab2-task1> docker run -d --name lab2-container -p 3000:3000 lab2-docker-app:v1
9ee7b6c8a126b89f8c8898c50b56fa3140b7beee587da790671ba46d1a72709c
• PS D:\Desktop\lab2-task1> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
9ee7b6c8a126   lab2-docker-app:v1  "docker-entrypoint.s..."  2 minutes ago  Up 2 minutes  0.0.0.0:3000-
>3000/tcp, [::]:3000->3000/tcp  lab2-container
• PS D:\Desktop\lab2-task1> docker logs lab2-container
Server running on port 3000
• PS D:\Desktop\lab2-task1> docker stop lab2-container
lab2-container
• PS D:\Desktop\lab2-task1> docker start lab2-container
lab2-container
• PS D:\Desktop\lab2-task1> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
9ee7b6c8a126   lab2-docker-app:v1  "docker-entrypoint.s..."  6 minutes ago  Up 9 seconds  0.0.0.0:3000-
>3000/tcp, [::]:3000->3000/tcp  lab2-container
• PS D:\Desktop\lab2-task1>
```

Figure 2. Original terminal evidence showing docker history for lab2-docker-app:v1.

This provides the required documentation of the image layers generated by the Dockerfile and inherited from the base image.

3. Running the Docker Container

The image was run as a detached container named **lab2-container** with host port 3000 mapped to container port 3000.

A terminal window with a dark background and light blue text. At the top, there is a link 'Focus folder in explorer (ctrl + click)' and a long alphanumeric string '0b7beee587da790671ba46d1a72709c'. Below this, the command 'docker ps' has been executed. The output is a table with columns: CONTAINER ID, IMAGE, COMMAND, CREATED, STATUS, and PORTS. One container is listed with ID '9ee7b6c8a126', image 'lab2-docker-app:v1', command '"docker-entrypoint.s..."', created '2 minutes ago', status 'Up 2 minutes', and ports '0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp'. The container name 'lab2-container' is shown at the end of the row. The prompt 'PS D:\Desktop\lab2-task1>' is visible at the bottom.

```
9ee7b6c8a126 0b7beee587da790671ba46d1a72709c
PS D:\Desktop\lab2-task1> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
9ee7b6c8a126   lab2-docker-app:v1  "docker-entrypoint.s..."  2 minutes ago  Up 2 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
lab2-container
PS D:\Desktop\lab2-task1>
```

Figure 3. Original terminal evidence showing **docker run -d --name lab2-container -p 3000:3000 lab2-docker-app:v1** and the running container.

The Docker port mapping is shown as 0.0.0.0:3000→3000/tcp, confirming that the application is published on host port 3000.

4. Application Verification in the Browser

After the container was started, the published application was opened in the browser. The expected HTML response was displayed.

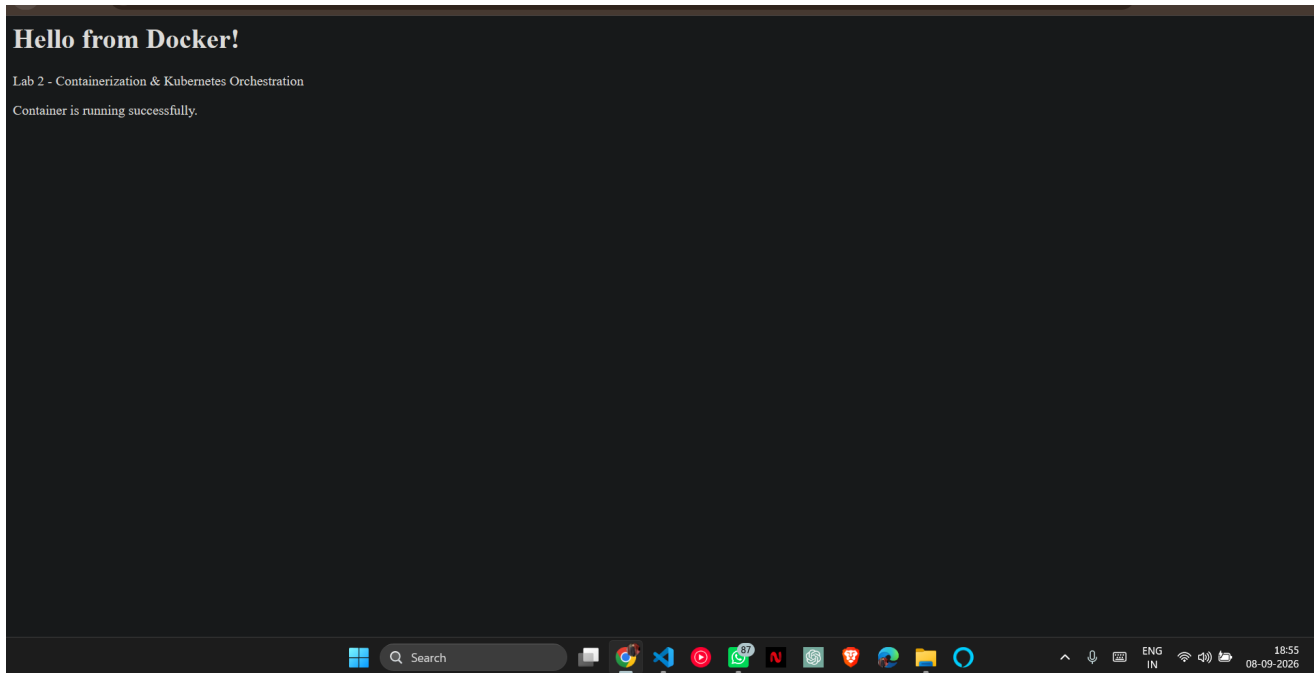
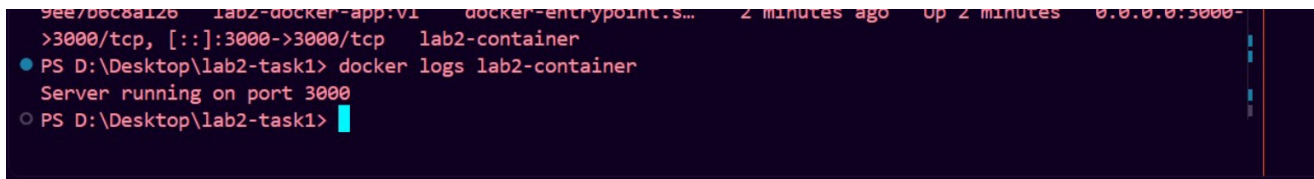


Figure 4. Original browser evidence showing "Hello from Docker!" and "Container is running successfully."

The browser response confirms that the Node.js application is reachable through the Docker-published port.

5. Container Logs and Runtime Evidence

The container logs were checked using **docker logs lab2-container**. The application printed its startup message, confirming successful server initialization.

A terminal window with a dark background and light-colored text. The top line shows a Docker container status: '9ee7b6c8a126 lab2-docker-app.v1 docker-entrypoint.s... 2 minutes ago Up 2 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp lab2-container'. Below this, a blue prompt character is followed by the command 'PS D:\Desktop\lab2-task1> docker logs lab2-container'. The output of the command is 'Server running on port 3000'. A second prompt character is followed by 'PS D:\Desktop\lab2-task1>' and a cursor.

```
9ee7b6c8a126 lab2-docker-app.v1 docker-entrypoint.s... 2 minutes ago Up 2 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp lab2-container
● PS D:\Desktop\lab2-task1> docker logs lab2-container
Server running on port 3000
○ PS D:\Desktop\lab2-task1>
```

Figure 5. Original terminal evidence showing the running container and **docker logs lab2-container** output.

Observed log: **Server running on port 3000.**

6. Task 1 Requirement Checklist

Requirement	Evidence / Result	Status
Write a Dockerfile	Dockerfile present and used for the build.	Completed
Build the image	docker build completed; image tagged lab2-docker-app:v1.	Completed
Run the container	lab2-container started with port 3000 published.	Completed
Document image layers	docker history lab2-docker-app:v1 captured image layers.	Completed
Document container behavior	docker ps, browser response and docker logs verify runtime behavior.	Completed

Conclusion

Task 1 was completed by containerizing the Node.js sample application with Docker, building and inspecting the image, running the container with port mapping, and verifying the application through browser output and container logs. The figures in this corrected report correspond to the evidence described in each section.

Submission note: Upload this PDF as the single file for Task 1. Keep the project source files in the GitHub repository separately.